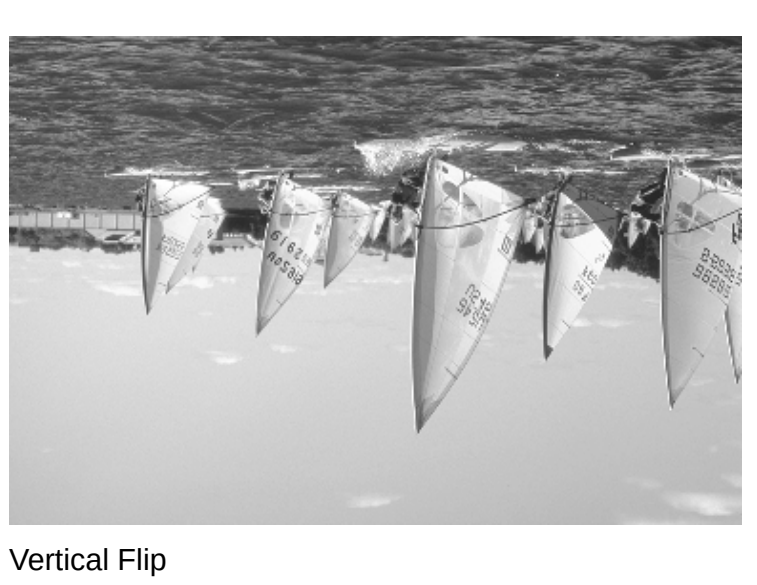
Lab 5

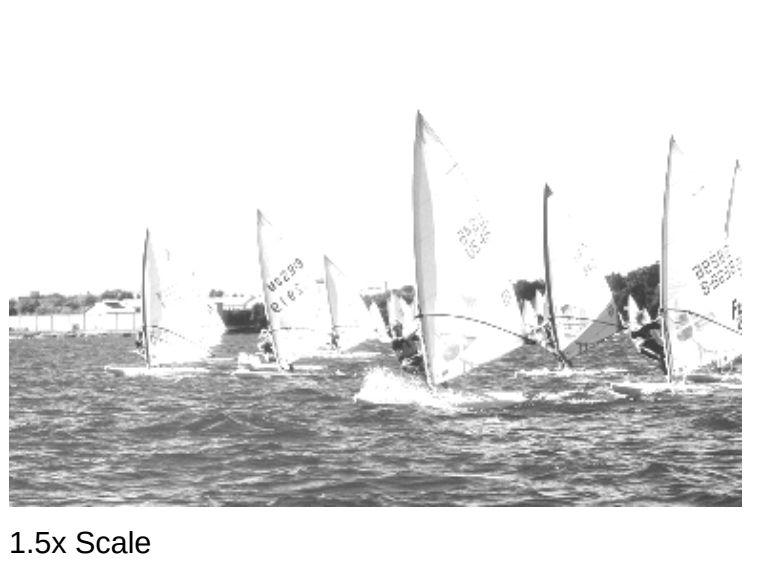
16.2.1 Exercise

Horizontal Flip



Vertical Flip

Negative

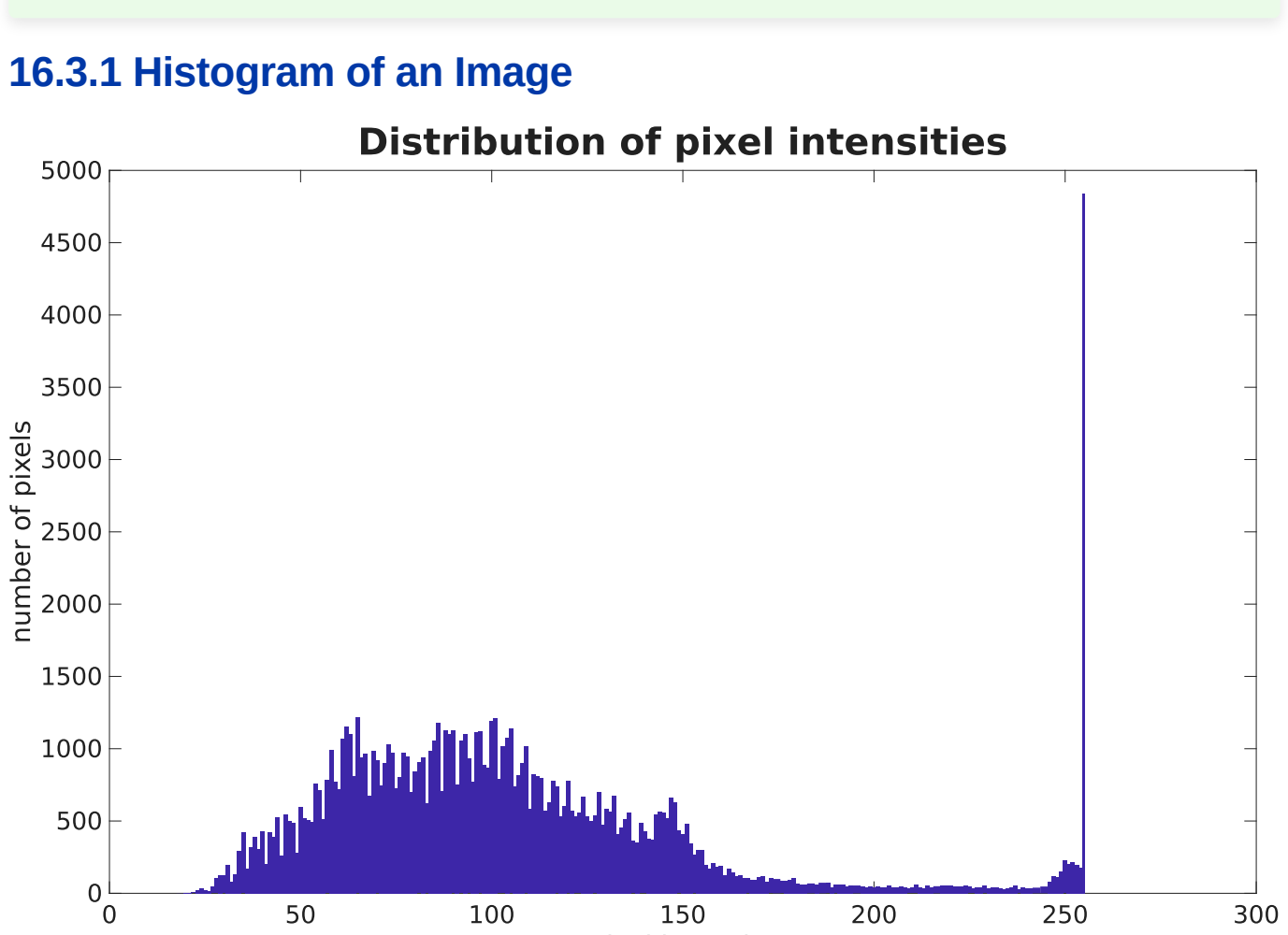


1.5x Scale

✓ Answer

For the 1.5x scaled image, the exposure of the image got blown out, clipping many of the pixels to their maximum value (255).

16.3.1 Histogram of an Image



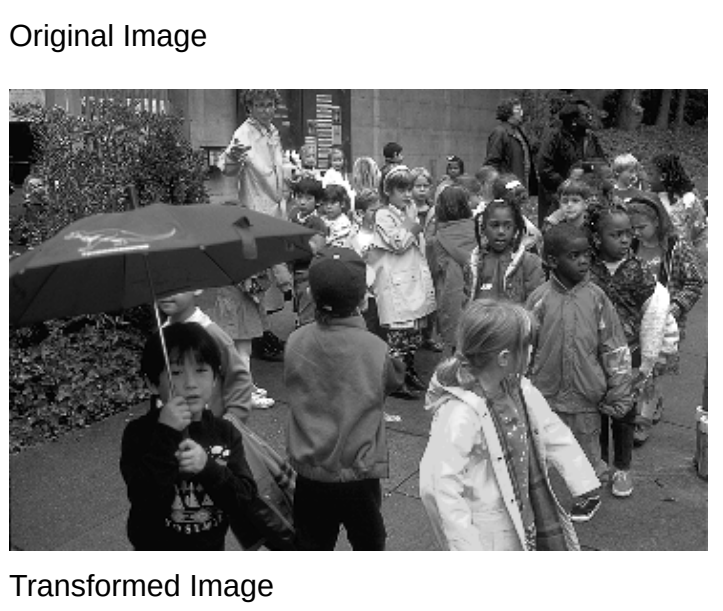
Histogram of house.tif

✓ Answer

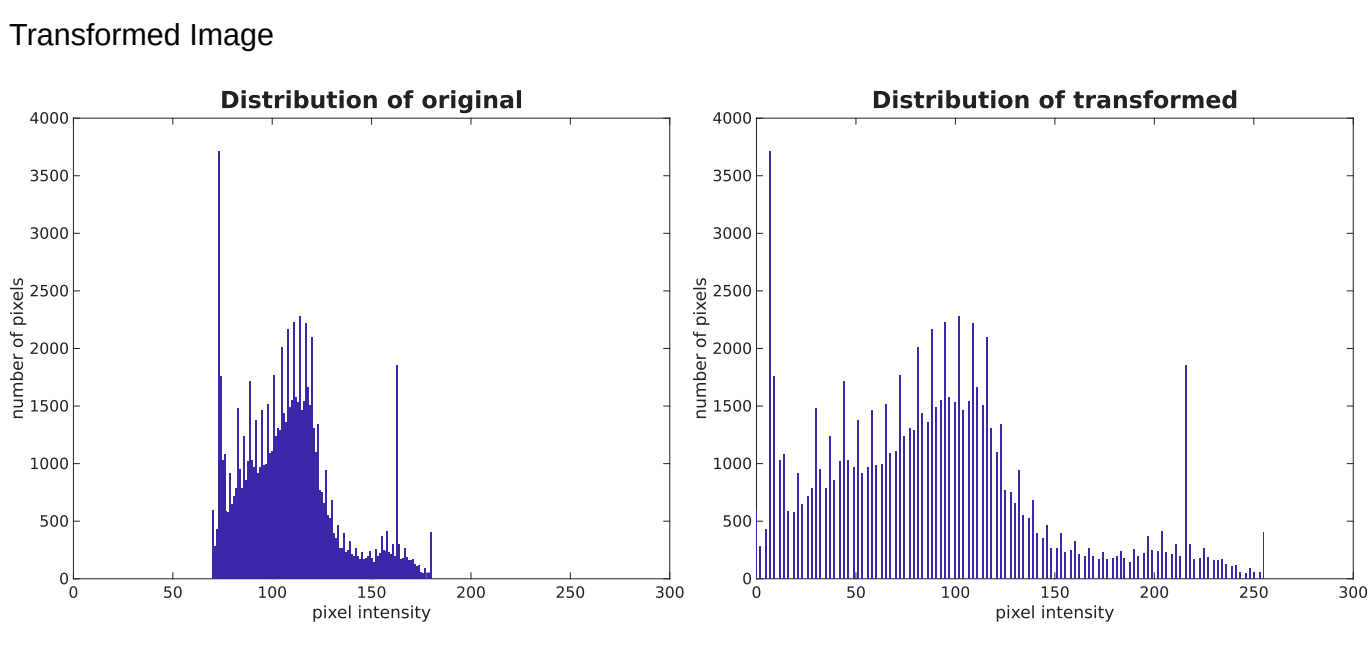
There is a fairly normal distribution of pixel intensities centered around 80. With a small congregation of outliers around 250, with a massive spike at 255.

16.3.2 Pointwise Transformations

Original Image



Transformed Image



Histogram of original (left) and transformed (right) image

✓ Answer

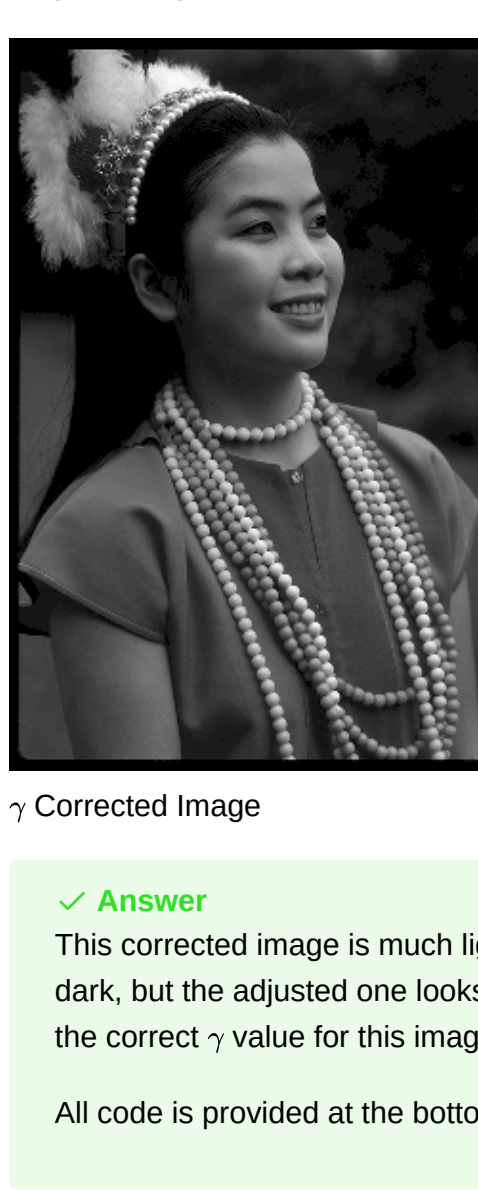
The transformation removed the visible gray haze from the image giving the transformed image much more contrast and making it easier to look at and digest. No visible negative effects from the filter can be seen.

The histograms appear to have a similar shape, but with the output histogram scaled horizontally. The reason we have zeros in the output is due to the fact that all the inputs are perfect integers, meaning adjacent input integers will have non-adjacent outputs due to being scaled by some amount. For example for a scaling of 1.5x, the inputs of 4 and 5 would map to 6 and 7.5=>8, which are non-adjacent.

All code is provided at the bottom of this document.

16.4 Gamma Correction

Original image



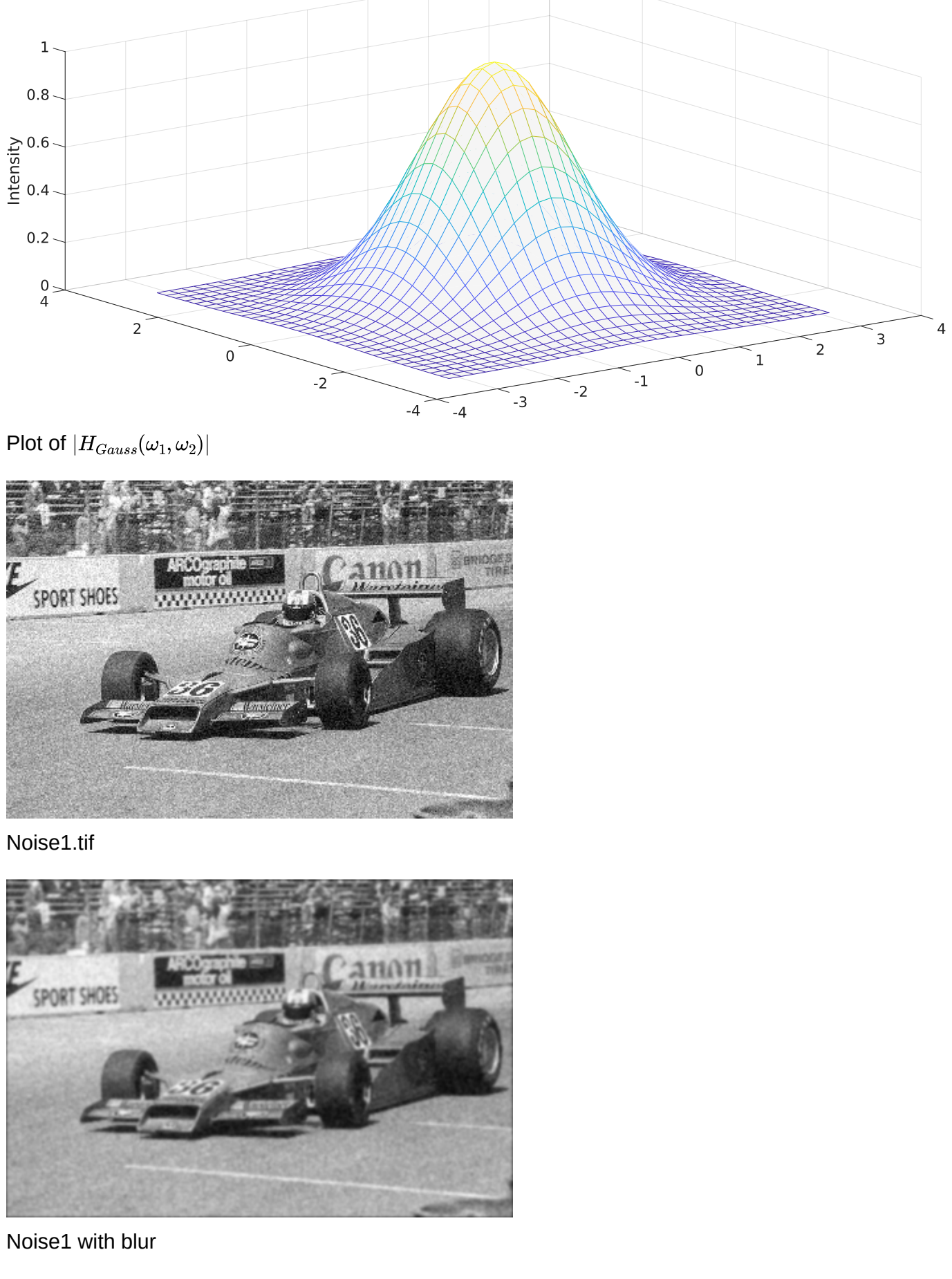
γ Corrected Image

✓ Answer

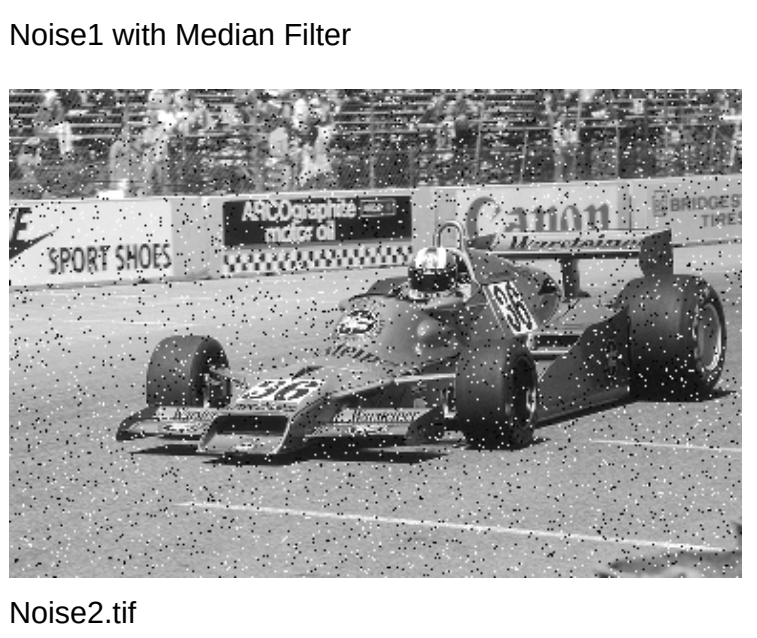
This corrected image is much lighter than the original image. The original image was very dark dark, but the adjusted one looks much more correct in terms of lighting. This does seem to be the correct γ value for this image.

All code is provided at the bottom.

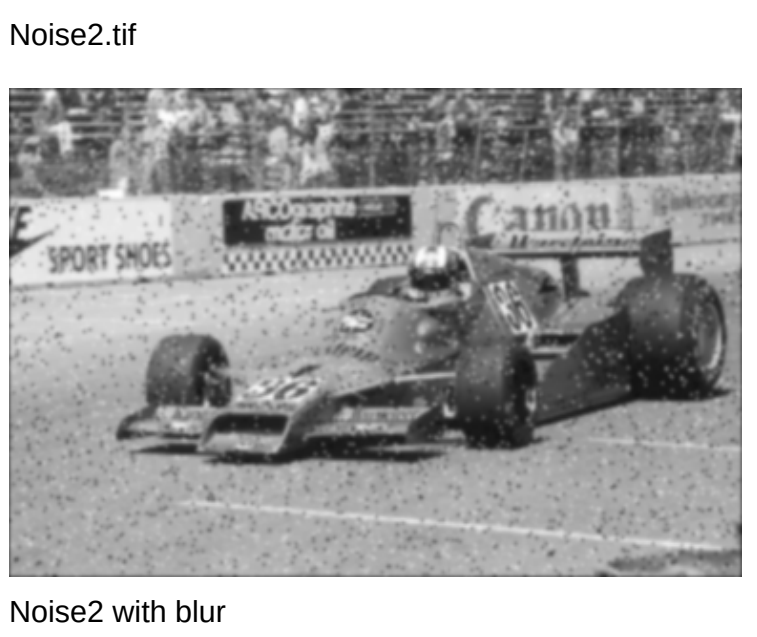
16.5.2 Smoothing Exercise



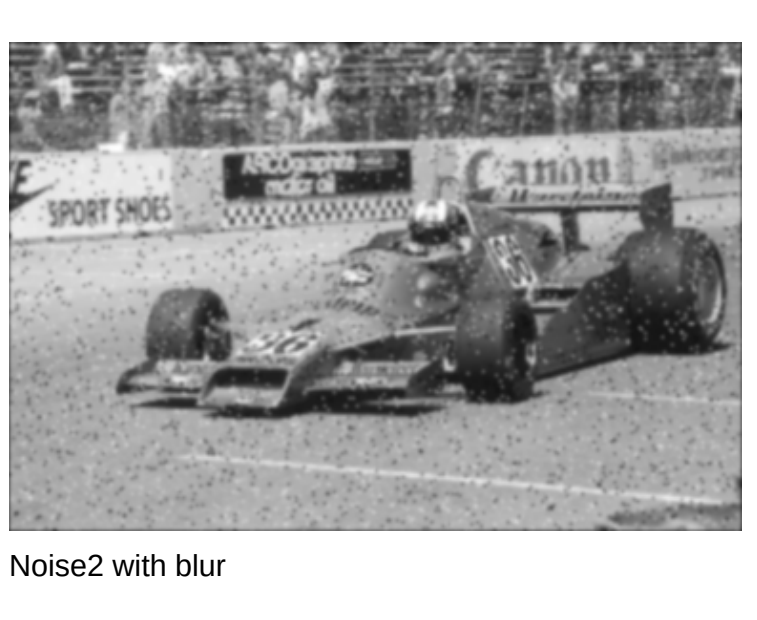
Plot of $|H_{Gauss}(\omega_1, \omega_2)|$



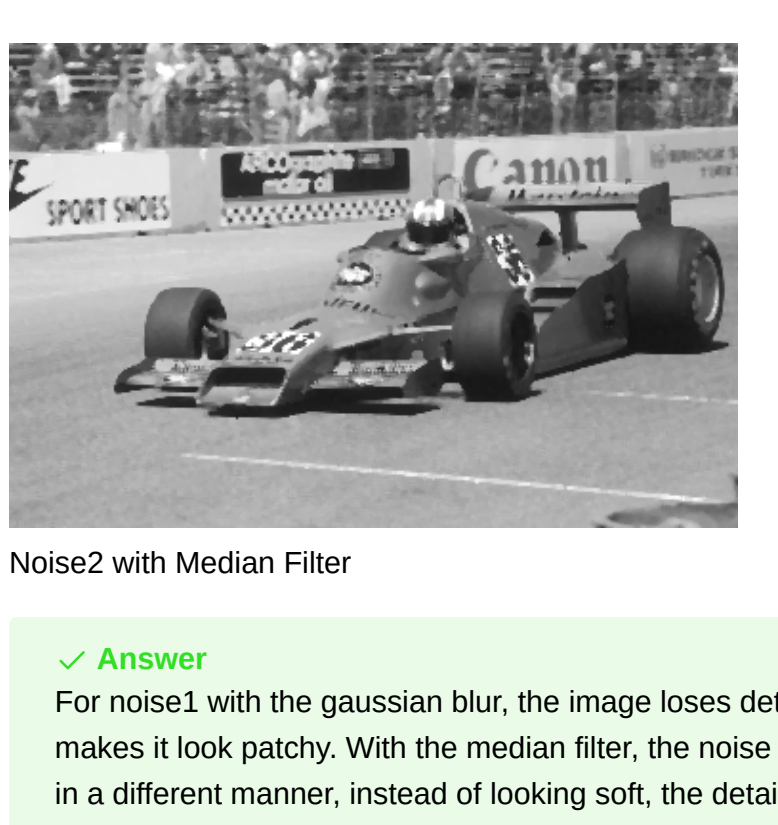
Noise1.tif



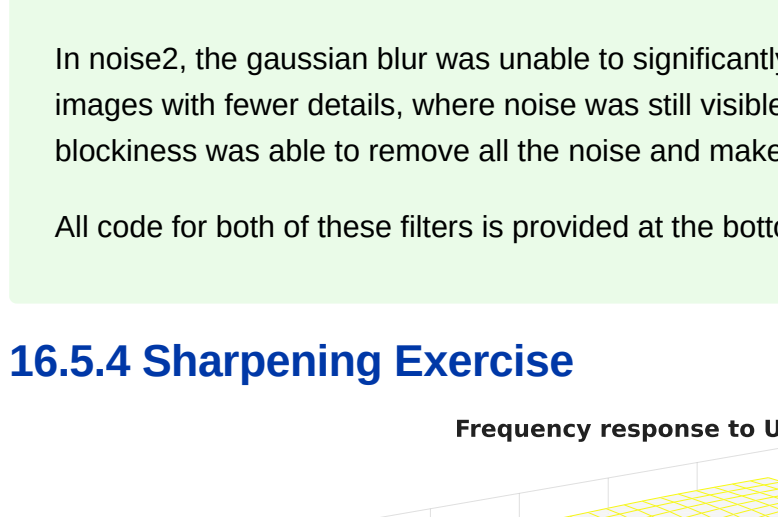
Noise1 with blur



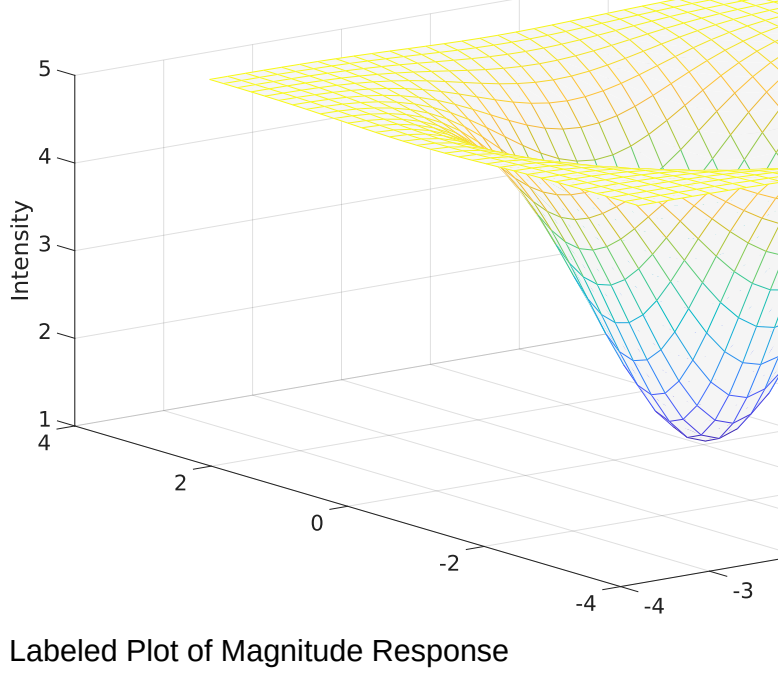
Noise1 with Median Filter



Noise2.tif



Noise2 with blur



Noise2 with Median Filter

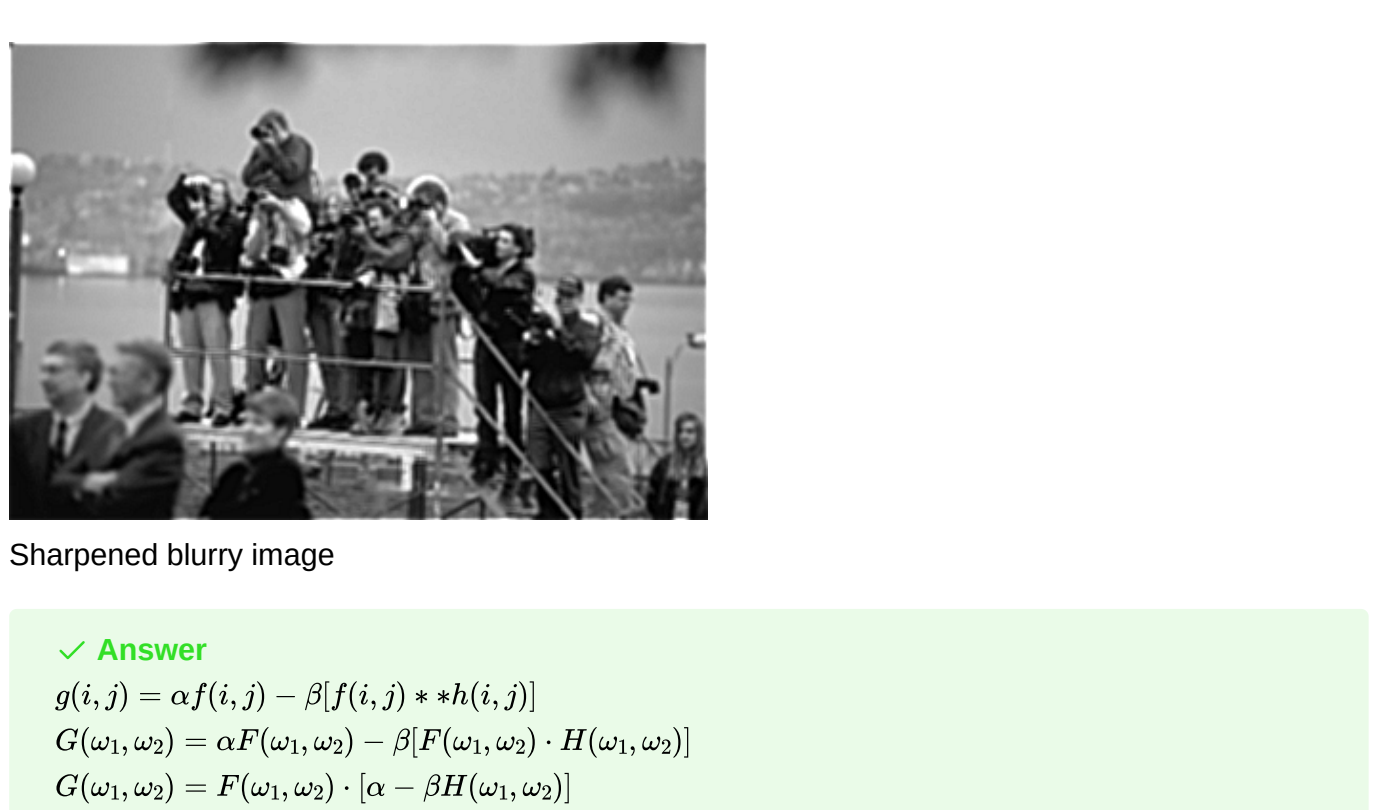
✓ Answer

For noise1 with the gaussian blur, the image loses details and slightly reduces the noise, but it makes it look patchy. With the median filter, the noise is reduced but the details also suffer but in a different manner, instead of looking soft, the details instead look blocky.

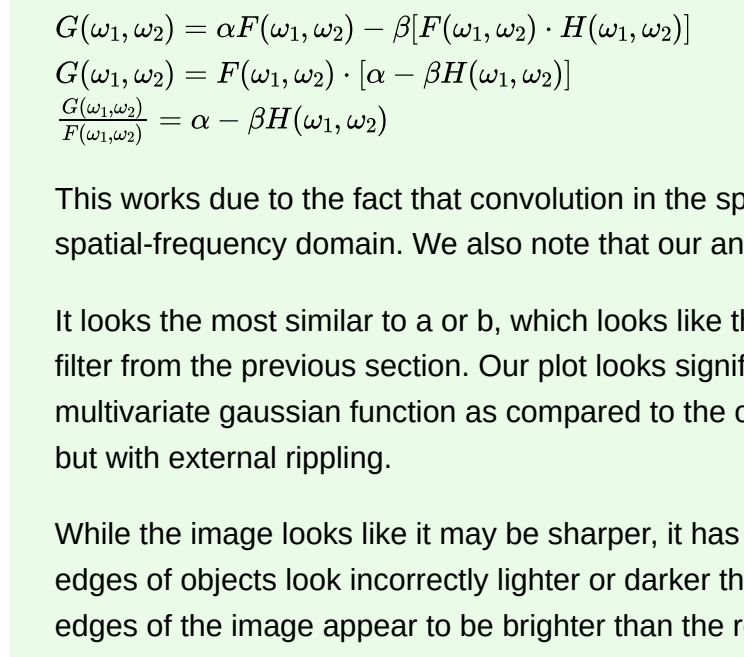
In noise2, the gaussian blur was unable to significantly reduce the noise, leading to a muted images with fewer details, where noise was still visible. The median filter, while with some blockiness was able to remove all the noise and make the image look cleaner.

All code for both of these filters is provided at the bottom.

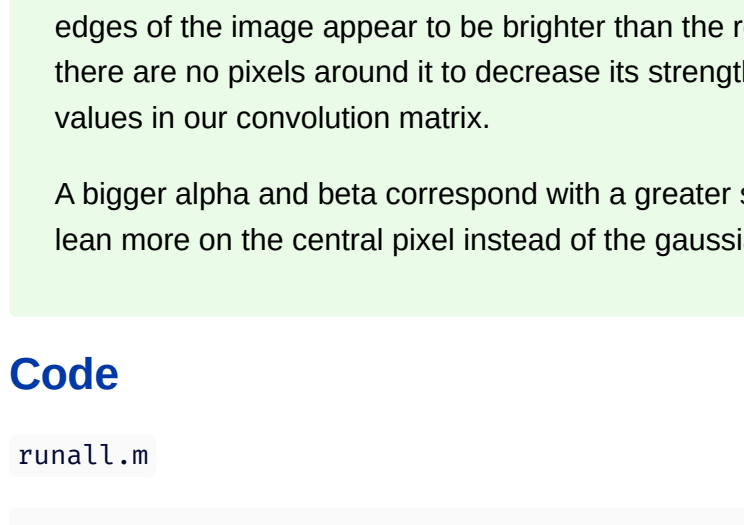
16.5.4 Sharpening Exercise



Labeled Plot of Magnitude Response



Blur



Sharpened blurry image

✓ Answer

$g(i,j) = \alpha f(i,j) - \beta[f(i,j) **h(i,j)]$
$G(\omega_1,\omega_2) = \alpha F(\omega_1,\omega_2) - \beta[F(\omega_1,\omega_2) \cdot H(\omega_1,\omega_2)]$
$G(\omega_1,\omega_2) = F(\omega_1,\omega_2) \cdot [\alpha - \beta H(\omega_1,\omega_2)]$
$\frac{G(\omega_1,\omega_2)}{F(\omega_1,\omega_2)} = \alpha - \beta H(\omega_1,\omega_2)$

This works due to the fact that convolution in the spatial domain becomes multiplication in the spatial-frequency domain. We also note that our analytical response fits this derivation directly.

It looks the most similar to a or b, which looks like the vertically flipped version of our low pass filter from the previous section. Our plot looks significantly more like a vertically flipped multivariate gaussian function as compared to the ones provided, which appear to be similar but with external rippling.

While the image looks like it may be sharper, it has an issue with "haloing" which is when the edges of objects look incorrectly lighter or darker than they should be. We also note that the edges of the image appear to be brighter than the rest, which we believe is due to the fact that there are no pixels around it to decrease its strength as those relevant pixels have negative values in our convolution matrix.

A bigger alpha and beta correspond with a greater sharpening effect of the output image, as we lean more on the central pixel instead of the gaussian.

Code

`runall.m`

```
% Clean workspace
close all
clear all

% Define working functions
functions = { @q1621, @q1631, @q1632, @q1640, @q1652 };

% Clean after each function
for i=1:numel(functions)
    functions{i}();
    clearvars -except functions i
end

close all
clear all
```

`helpers.m`

```
% A function to help set our defaults for all plots
function [fontSize, export] = helpers(f, varargin)
    p = inputParser;

    % Define defaults
    addParameter(p, 'w', 1000);
    addParameter(p, 'h', 800);
    parse(p, varargin{:});

    % Disables LaTeX rendering, as it is not supported by the export function
well
    set(f,'defaultTextInterpreter','none')
    set(f,'defaultAxesTickLabelInterpreter','none')
    set(f,'defaultLegendInterpreter','none')
    % Increase Default font size
    defaultFontSize = 25;
    set(f, 'defaultTextFontSize', defaultFontSize);
    set(f, 'defaultAxesFontSize', defaultFontSize);
    set(f, 'defaultLegendFontSize', defaultFontSize);

    % Set Font
    set(f, 'DefaultAxesFontName', 'Arial');
    set(f, 'DefaultTextFontName', 'Arial');
    set(f, 'DefaultLegendFontName', 'Arial');
    set(f, 'DefaultColorbarFontName', 'Arial');

    % Set clear background
    set(f, 'DefaultFigureColor', 'none');
    set(f, 'DefaultAxesColor', 'none');
    set(f, 'DefaultLegendColor', 'none');
    set(f, 'DefaultFigureInvertHardcopy', 'off');

    % Set the default layout to "loose"
    set(f, 'DefaultTiledLayoutTileSpacing', 'loose');
    set(f, 'DefaultTiledLayoutPadding', 'loose');

    % Increase Canvas size
    set(f, 'Position', [0 0 p.Results.w p.Results.h]);

    % Set text alignment properties
    set(f, 'DefaultAxesTitleHorizontalAlignment', 'center');
    set(f, 'DefaultTextHorizontalAlignment', 'center');

    % Disable slow output warning
    warning('off', 'MATLAB:print:ContentTypeImageSuggested')

    % Set high quality output by default
    set(f, 'DefaultFigureRenderer', 'painters');
    f.Visible = "on";
    % Return our font size
    fontSize = defaultFontSize;
    export = @(fp) exportgraphics(f, fp, ContentType='vector',
BackgroundColor='none');
end
```

`q1621.m`

```
% Read image and reexport it
yacht = imread("images/yacht.tif");
imwrite(yacht, "out_images/1621_yacht.png");

% Invert the image
inv = 255 - yacht;
imwrite(inv, "out_images/1621_yacht_inv.png");

% Flip the image
horflip = fliplr(yacht);
imwrite(horflip, "out_images/1621_yacht_horflip.png");

% Flip it again
horflip = flipud(yacht);
imwrite(horflip, "out_images/1621_yacht_verflip.png");

% Scale all the pixels (over 255 gets automatically clipped)
scaled = yacht * 1.5;
imwrite(scaled, "out_images/1621_yacht_scaled.png");
```

`q1631.m`

```
% Read input image
house = imread("images/house.tif");

% Flatten the array for plotting
house_r = reshape(house, 1, []);

% Set up plot
f = figure;
[ff, e] = helpers(f, w=2000, h=1200);

% Plot histogram
hist(house_r, 0:255);

% Label the plot
xlabel("pixel intensity");
ylabel("number of pixels");

title("Distribution of pixel intensities", FontSize=ff*1.5);

% Export
e("plots/1631_hist.svg");
```

`q1632.m`

```
% Transform the inputs from T1 to T2 to the scale of 0 to 255
pointTrans = @(input, T1, T2) uint8(min(max((double(input) - T1) ./ (T2 - T1)
.* 255, 0), 255));

% Read input image
narrow = imread("images/narrow.tif");
imwrite(narrow, "out_images/1632_narrow.png");

% Apply our filter
narrow_trans = pointTrans(narrow, 70, 180);

% Export image
imwrite(narrow_trans, "out_images/1632_narrow_scale.png");

% Plot both histograms of input and output
f = figure;
[ff, e] = helpers(f, w=3000, h=1200);
t = tiledlayout(f, 1, 2);

nexttile;

hist(reshape(narrow, 1, []), 0:255);
xlabel("pixel intensity");
ylabel("number of pixels");
title("Distribution of original", FontSize=ff*1.5);

nexttile;

hist(reshape(narrow_trans, 1, []), 0:255);
xlabel("pixel intensity");
ylabel("number of pixels");
title("Distribution of transformed", FontSize=ff*1.5);

% Export plot
e("plots/1632_hist.svg");
```

`q1640.m`

```
% Our gamma correction function
gammCorr = @(input, gamma) uint8(255 .* ((double(input) ./ 255) .^ (1/gamma)));

% Read input function
dark = imread("images/dark.tif");
imwrite(dark, "out_images/1640_dark.png");

% Apply out filter
dark_corr = gammCorr(dark, 2.2);

% Export the image
imwrite(dark_corr, "out_images/1640_dark_corr.png");
```

`q1652.m`

```
% Define out gaussian function
h = @(v) @(i) exp(- i^2 / 2 / v);
% Define the range to eval the gaussian given an N
r = @(N) (-(N-1)/2):1:((N-1)/2);
% Normalize matrix equation
nor = @(A) A / sum(A);
% Outer product of a vector
outer = @(A) A' * A;
% Out gaussian filter
gaussFilter = @(N, v) outer(nor(arrayfun(h(v), r(N))));
% Our median filter (performs a sliding window and takes median)
medianFilter = @(X, N) reshape(median(im2col(X, [N, N], 'sliding')), [size(X,
1) - N + 1, size(X, 2) - N + 1]);
% Our unsharp filter (a * impulse - b * gaussian)
unsharp = @(N, v, a, b)  a * padarray(1, [(N-1)/2, (N-1)/2]) - b *
gaussFilter(N, v);

% Define variables we are using
N = 7;
variance = 1;
DFT_size = 32;
a = 5;
b = 4;

% Create our gaussian filter
h_matrix = gaussFilter(N, variance);
H = fftshift(fft2(h_matrix, DFT_size, DFT_size));
H_mag = abs(H);

omega = linspace(-pi, pi, DFT_size);
[W1, W2] = meshgrid(omega, omega);

% Plot its frequency response from -pi to pi
f = figure;
[ff, e] = helpers(f, w=2500, h=1200);

mesh(W1, W2, H_mag);
zlabel("Intensity");
title("Frequency response to Gaussian Filter");

% Export plot
e("plots/1652_fft.svg");

% Read input image
noise1 = imread("images/noise1.tif");
imwrite(noise1, "out_images/1652_race_noise1.png");

% Apply our filters
noise1_blur = uint8(filter2(h_matrix, noise1));
noise1_med = medianFilter(noise1, 3);

% Export images
imwrite(noise1_blur, "out_images/1652_race_noise1_blur.png");
imwrite(noise1_med, "out_images/1652_race_noise1_med.png");


% Read second image
noise2 = imread("images/noise2.tif");
imwrite(noise2, "out_images/1652_race_noise2.png");

% Apply filters again
noise2_blur = uint8(filter2(h_matrix, noise2));
noise2_med = medianFilter(noise2, 3);

% Export images
imwrite(noise2_blur, "out_images/1652_race_noise2_blur.png");
imwrite(noise2_med, "out_images/1652_race_noise2_med.png");


% Create our unsharp filter
h_matrix = unsharp(N, variance, a, b);
H = fftshift(fft2(h_matrix, DFT_size, DFT_size));
H_mag = abs(H);

omega = linspace(-pi, pi, DFT_size);
[W1, W2] = meshgrid(omega, omega);

% Plot the response of the unsharp filter
f = figure;
[ff, e] = helpers(f, w=2500, h=1200);

mesh(W1, W2, H_mag);
zlabel("Intensity");
title("Frequency response to Unsharp Filter");

% Export plot
e("plots/1654_unsharp.svg");

% Read input image
blur = imread("images/blur.tif");
imwrite(blur, "out_images/1654_blur.png");

% Apply filter
blur_sharp = uint8(filter2(h_matrix, blur));

% Export image
imwrite(blur_sharp, "out_images/1654_blur_sharp.png");
```